SMRTTECH 4AI3 - Artifical Intelligence and Machine Learning

Lab 5 - Multi-class classification & Cross-validation

Instructor: Siqi Zhao

Due: Midnight 11:59PM, day before your next lab session

Please read and follow these instructions:

- Submit your lab on Avenue to Learn (A2L) before the due date (midnight of the day before your next lab session). The submission will be closed after the deadline.
- Do your lab exercises on your own, i.e., do not copy. If two lab reports (or some parts of them) are found to be copies of each other, then BOTH shall receive 0 mark.
- You may use Google Colab or any other Python programming platform of your own choice to run your programs.
- Please read the questions carefully and use AI / machine learning libraries only when specially required as part of the question. Also add comments in your code where ever necessary.
- Submit your working python source files (or Jupyter notebooks) in zipped or source code format.

1 Exercises (20 Points)

1. **Load the dataset Balance Scale Data Set.**

   Balanced scale Dataset is from UCI repository which was generated to model psychological experimental results. Each example is classified as having the balance scale tip to the right, tip to the left, or be balanced. The attributes are the left weight, the left distance, the right weight, and the right distance. The correct way to find the class is the greater of (left-distance * left- weight) and (right-distance * right-weight). If they are equal, it is balanced.

   Attribute Information:

   - Left-Weight: 5 (1, 2, 3, 4, 5)
   - Left-Distance: 5 (1, 2, 3, 4, 5)
   - Right-Weight: 5 (1, 2, 3, 4, 5)
   - Right-Distance: 5 (1, 2, 3, 4, 5)
   - Class Name: 3 (L, B, R)

Here, label is categorical variable with labelled as Left balanced, balanced and right balanced.

2. **Merge Left and right columns**

   Create a new column LeftBalance with multiplication of Left-Weight and Left-Distance.

   Create a new column RightBalance with multiplication of Right-Weight and Right-Distance.

3. **Drop Left-Weight, Left-Distance, Right-Weight, Right-Distance columns.**

   At this stage you should have two features Left-Balance, Right-Balance and one label.

4. **Visualize the data**

   Generate a scatter plot for this 2-D data. Use assorted colors to differentiate data points for different classes. (Use two newly generated features)

5. **Split dataset to train and test data**

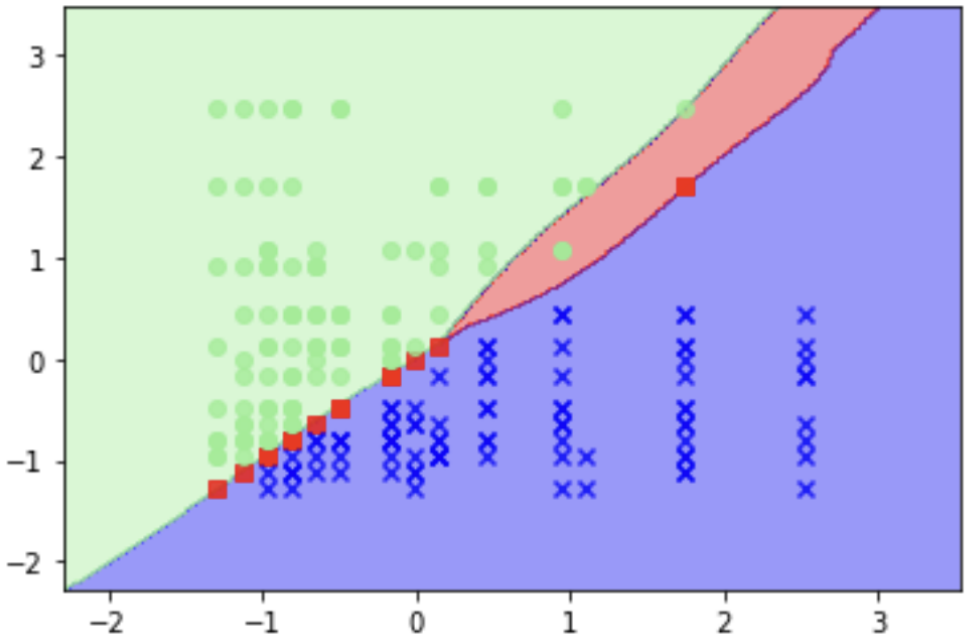
   Split the data points into training set and testing set using the train_test_split() method. Limit the size of the testing set to 20% with a fixed random state number.

6. **Normalized the data**

   Normalize the data using StandardScaler.

7. **Create visualize function**

   Create a visualization function which accepts Test dataset (X, y) and trained algorithm as an input and display contour around different classed and display classes boundaries. Your output should look like below figure.



8. **Train the dataset using Logistic Regression**

   a. Training

   Create three Logistic regression classifier models by using kernels of 'lbfgs', 'liblinear' and 'saga'. You can explore with different values for the hyperparameters to improve the performance. Fit the models with the training set and training labels. Print the scores for these three models on the test set.

b. Visualize the result

Visualize all the three algorithms result in above exercise using the function created at question 7

9. **Cross-Validation and Model Selection**

a. Cross-Validation

Use cross_val_score() from the sklearn.model_selection module to perform k-fold cross-validation (with k=5) on the Logistic Regression models. Report the average cross-validation score for each model.

b. Model Selection

Based on the cross-validation results, choose the best model and justify your selection. You can use both the cross-validation scores and the model's performance on the test set for your justification.